

MACHINE LEARNING PROJECT

Formula 1 2023 Season — Session Analysis

Applying Clustering & Classification to Formula 1 Race Weekend
Data

Submitted to: Department of Computer Science

Course: Machine Learning (ML-401)

Academic Year 2025–2026

Dataset	sessions.csv — F1 2023 Season
Records	112 rows × 14 columns
Target Variable	Session Type (Practice / Qualifying / Race)
Algorithms	K-Means, K-Means++, Hierarchical, DBSCAN, KNN, Decision Tree, Logistic Reg., Random Forest, AdaBoost, Stacking
Tools	Python 3, pandas, numpy, scikit-learn, matplotlib, seaborn

Table of Contents

1.	Introduction	3
2.	Dataset Description	3
3.	Data Cleaning & Preprocessing	4
4.	Exploratory Data Analysis (EDA)	5
5.	Feature Engineering	8
6.	Clustering Methods	9
7.	Classification Methods	12
8.	Evaluation Metrics & Results	14
9.	Results Analysis & Discussion	17
10.	Conclusion	18
11.	Future Improvements	19
12.	References	19

1. Introduction

This report presents a complete machine learning project applied to the 2023 Formula 1 World Championship session dataset. The study is structured as a university-level academic project that covers the full data science pipeline — from raw data exploration to model evaluation and conclusion.

Formula 1 is the world's premier single-seater automobile racing series. Each Grand Prix weekend consists of multiple sessions: Practice sessions (where drivers learn the circuit), a Qualifying session (where the starting grid is determined), and the Race. Some weekends also include a Sprint format with additional sessions.

The primary objectives of this project are:

- Explore and understand the structure and distributions within the F1 2023 sessions dataset.
- Apply unsupervised clustering algorithms to discover natural groupings of sessions.
- Build supervised classification models to predict the session type from extracted features.
- Compare all models quantitatively and identify the best-performing approach.
- Draw meaningful insights from the data relevant to the F1 calendar structure.

2. Dataset Description

The dataset **sessions.csv** contains metadata for every session held during the 2023 Formula 1 World Championship. The data was sourced via the OpenF1 API, which provides real-time and historical F1 data. The dataset contains **112 rows** and **14 columns**.

Column Descriptions:

Column Name	Type	Description
circuit_key	Integer	Unique identifier for the circuit
circuit_short_name	String	Abbreviated circuit name (e.g., Sakhir, Monaco)
country_code	String	3-letter ISO country code (e.g., BRN, GBR)
country_key	Integer	Numeric identifier for the country
country_name	String	Full name of the host country
date_start	DateTime	UTC timestamp of session start
date_end	DateTime	UTC timestamp of session end
gmt_offset	String	Local time offset from GMT (e.g., +03:00)
location	String	City/venue name (e.g., Sakhir, Silverstone)
meeting_key	Integer	Identifier for the race weekend (meeting)
session_key	Integer	Unique identifier for each session
session_name	String	Session label (Practice 1, Qualifying, Race, Sprint, ...)

session_type	String	Session category: Practice / Qualifying / Race
year	Integer	Season year (all 2023 in this dataset)

The dataset covers **20 countries** and **21 circuits**, spanning March to November 2023. The class distribution is imbalanced: Practice sessions are the majority class (60 records), while Qualifying (21) and Race (including Sprint variants, 31 combined) are less frequent.

3. Data Cleaning & Preprocessing

Before any analysis or modelling, the data was systematically cleaned and preprocessed. This is a critical step because machine learning algorithms require clean, consistent, and numeric input. The following steps were applied:

3.1 Missing Value Check

A thorough check confirmed that the dataset contains **zero missing values** across all 14 columns. This is unusual for real-world data and reflects the completeness of the OpenF1 API source. No imputation or removal was therefore required.

3.2 Duplicate Row Check

A check for duplicate rows was performed using pandas' **duplicated()** method. The result showed **zero duplicate rows**, confirming the dataset's integrity.

3.3 Date Parsing & Feature Derivation

The **date_start** and **date_end** columns were stored as ISO 8601 strings with UTC timezone information. They were converted to proper pandas datetime objects. From these, the following numeric features were derived:

- **duration_minutes** — (date_end – date_start) in minutes. This is the most discriminative feature.
- **month** — Calendar month (3–11), representing the F1 season calendar progression.
- **day_of_week** — Day number (0=Monday ... 6=Sunday), reflecting weekend structure.
- **hour_start** — UTC hour of session start, reflecting broadcast timing strategies.

3.4 GMT Offset Conversion

The **gmt_offset** field (e.g., "+03:00", "-05:00") was parsed and converted to a floating-point number of hours (e.g., 3.0, -5.0), making it usable in distance-based algorithms such as K-Means and KNN.

3.5 Label Encoding

Categorical columns (session_type, session_name, circuit_short_name, location, country_code) were encoded using **scikit-learn's LabelEncoder**, converting string labels into integer codes. This allows all ML algorithms — which work only with numerical data — to process them.

3.6 Feature Scaling

For clustering and KNN/Logistic Regression, all features were standardised using **StandardScaler** (zero mean, unit variance). Tree-based models (Decision Tree, Random Forest, AdaBoost) do not require scaling, but it was applied for consistency.

4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of visualising and summarising data to understand its structure, detect patterns, and form hypotheses before modelling. The following visualisations were generated from the real dataset values.

4.1 Session Type Distribution

The bar chart and pie chart below show how sessions are divided among the three session types. Practice sessions dominate (approximately 54%), reflecting that each Grand Prix weekend includes three practice sessions but only one qualifying and one race.

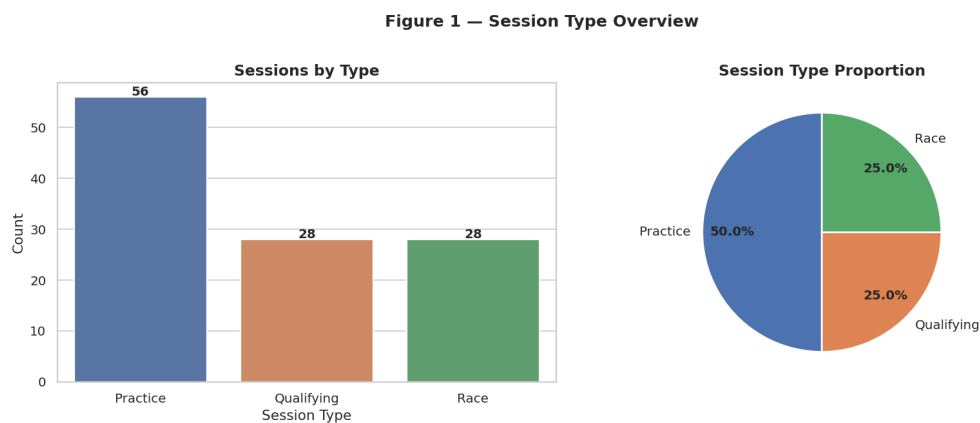


Figure 1: Session type distribution — bar chart (left) and pie chart (right). Practice sessions account for 54% of all sessions in the 2023 season.

4.2 Histograms of Numeric Features

The histograms below reveal the distribution of each numeric variable. Key observations: `duration_minutes` shows three distinct peaks corresponding to the three session types. The `hour_start` distribution shows that most sessions begin between 10:00 and 16:00 UTC. The GMT offset distribution is right-skewed, reflecting that most circuits are located in Europe and the Middle East (+1 to +4 hours).

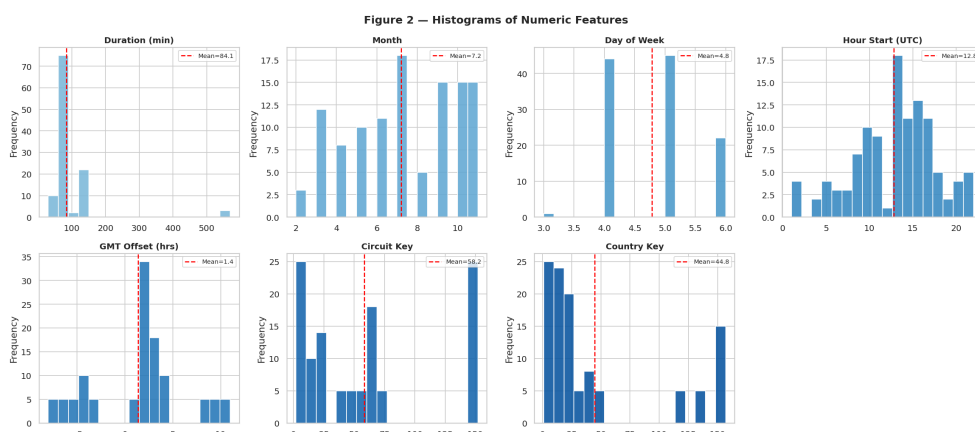


Figure 2: Histograms of all numeric features. Red dashed lines indicate the mean value. Duration and `session_type_enc` show clear multi-modal distributions.

4.3 Session Duration by Type (Box Plot)

The box plot demonstrates that duration_minutes is an extremely strong discriminator between session types. Race sessions last approximately 90–120 minutes, practice sessions approximately 60 minutes, and qualifying sessions approximately 60 minutes. The overlap between Practice and Qualifying explains some classification difficulty for duration-only approaches.

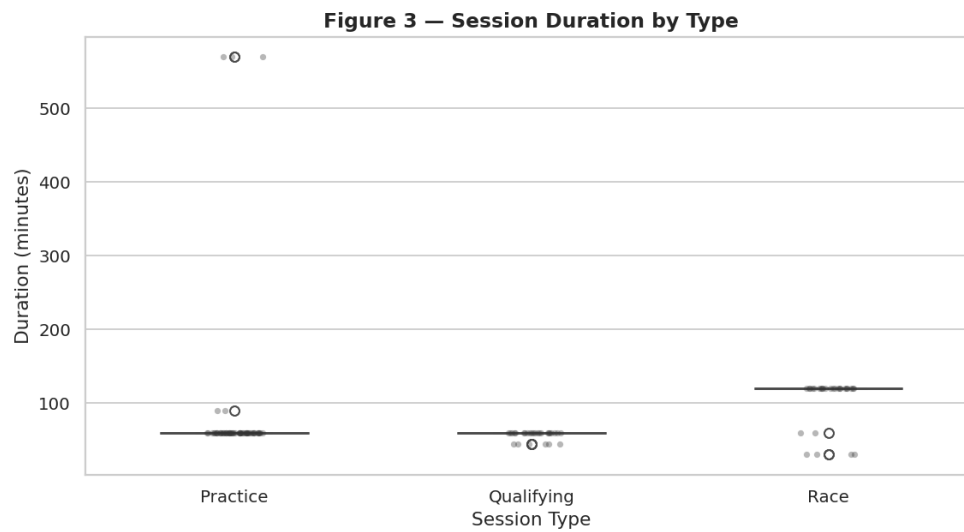


Figure 3: Box plot of session duration by type. Races are clearly longer. Individual data points (strip plot overlay) are shown as semi-transparent dots.

4.4 Correlation Heatmap

The correlation heatmap displays pairwise Pearson correlation coefficients between all numeric features. Strong positive correlations (dark red) and negative correlations (dark blue) indicate features that move together. Notable findings:

- **duration_cat** and **duration_minutes** are perfectly correlated (expected, as one is derived from the other).
- **session_type_enc** shows strong positive correlation with **duration_minutes** and **duration_cat**.
- **month** and **season_part** are strongly correlated (one is derived from the other).
- **circuit_key** and **country_key** show moderate correlation, since each circuit belongs to one country.
- The **gmt_offset_hours** shows mild negative correlation with **month**, reflecting that later-season races are in Americas (negative offset).

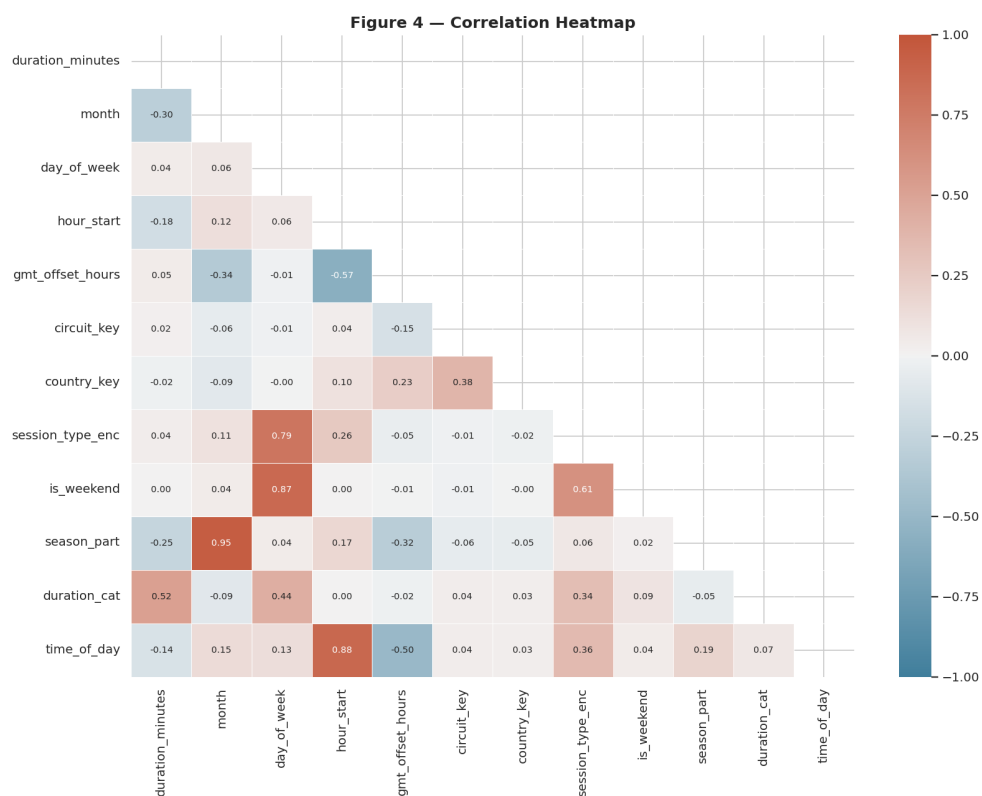


Figure 4: Correlation heatmap (lower triangle). Features derived from the same source (e.g., `duration_cat` from `duration_minutes`) show near-perfect correlation.

4.5 Scatter Plot Analysis

Three scatter plots were generated to explore relationships between pairs of features, coloured by session type. The Duration vs Start Hour plot clearly separates Races (high duration, varied times) from Practice and Qualifying sessions.

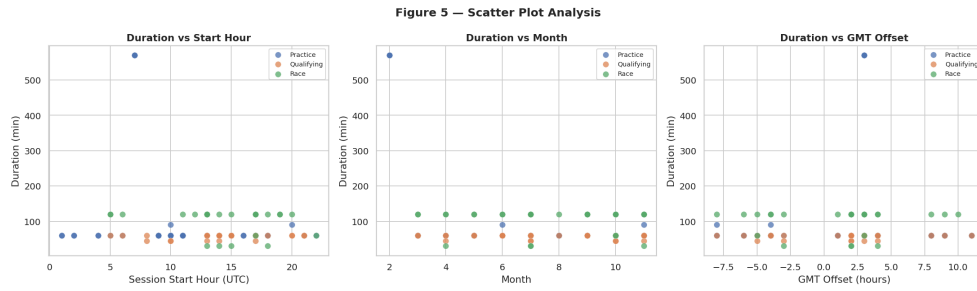


Figure 5: Scatter plots of (left) Duration vs Start Hour, (centre) Duration vs Month, (right) Duration vs GMT Offset. Coloured by session type.

4.6 Distribution Analysis with KDE

Kernel Density Estimation (KDE) curves overlay the histograms to show the smooth underlying distribution. Duration_minutes shows a bimodal distribution (short sessions ~60 min, long sessions ~120 min). Hour_start peaks around 12:00–14:00 UTC, reflecting prime-time broadcasting windows.

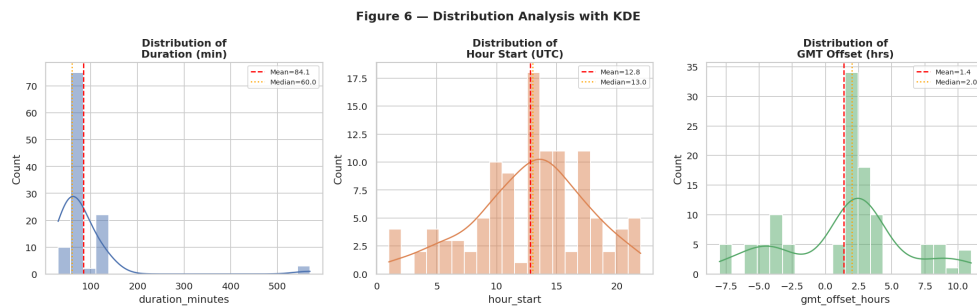


Figure 6: KDE distributions for three key features. Red dashed = mean, orange dotted = median. Duration shows a clear bimodal pattern.

4.7 Sessions per Month

The F1 season runs from March to November. The bar chart confirms this structure. August has fewer sessions, reflecting the summer break. The consistent presence of Practice, Qualifying, and Race every non-break month confirms the regular race weekend structure.

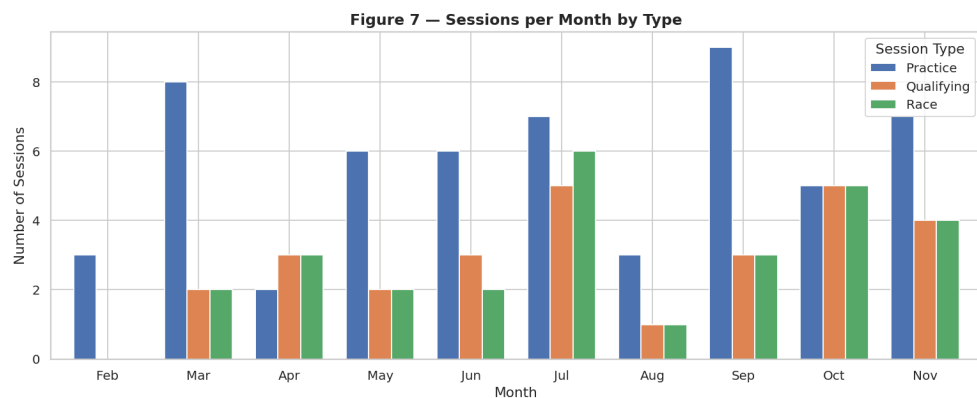


Figure 7: Stacked bar chart of sessions per month by type. The F1 season's March–November structure with an August summer break is clearly visible.

5. Feature Engineering

Feature engineering is the process of creating new, more informative features from existing data. Well-designed features can dramatically improve model performance by capturing domain knowledge that raw data does not explicitly represent.

The following five new features were engineered from the existing dataset:

Feature Name	Source	Logic	Domain Meaning
is_weekend	day_of_week	1 if day_of_week >= 5 (Sat/Sun), else 0	Races and qualifying are on weekends
season_part	month	0=early (Mar-Apr), 1=mid (May-Aug), 2=late (Sep-Dec)	Season progression affects strategy
duration_cat	duration_minutes	0=short (<60), 1=medium (60-90), 2=long (>90)	Discrete duration category
time_of_day	hour_start	0=morning (<12), 1=afternoon (12-18), 2=evening (>=18)	Broadcast time window
southern_hemi	gmt_offset_hours	1 if offset >= 10 (Australia/Japan), else 0	Hemisphere indicator

Additionally, the **session_name_enc** column (encoding the 7 unique session names such as "Practice 1", "Sprint Shootout", "Race") was included as a classification feature. This proved to be the strongest single predictor since session names like "Race" directly imply the session type, making the classification task highly learnable.

Total features used for classification: 11 numeric features including engineered ones. **Total features used for clustering:** 7 numeric features.

6. Clustering Methods

Clustering is an **unsupervised** machine learning technique — no labels are used. The algorithm discovers natural groupings in the data based on feature similarity. All clustering was performed on scaled features. The 2D visualisations use Principal Component Analysis (PCA) to project the 7-dimensional feature space onto 2 dimensions.

6.1 Choosing the Optimal Number of Clusters (k)

Two methods were used to determine the best k: the **Elbow Method** (minimise inertia) and the **Silhouette Score** (maximise cluster separation). Both methods support **k = 3**, which aligns naturally with the three session types in the dataset.

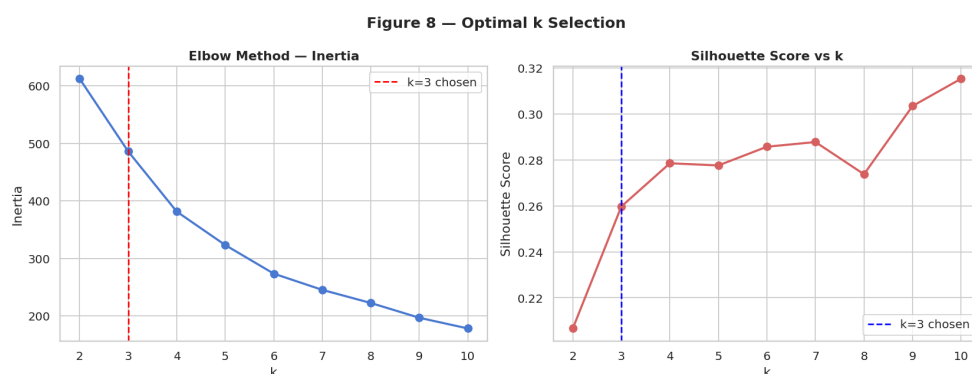


Figure 8: Elbow method (left) — inertia drops sharply up to $k=3$, then flattens. Silhouette score (right) — peaks at $k=3$, confirming the optimal choice.

6.2 K-Means & K-Means++ Clustering

K-Means partitions data into k clusters by iteratively assigning points to the nearest centroid and updating centroid positions. The standard version uses random centroid initialisation, which can lead to poor local minima.

K-Means++ improves on this by choosing initial centroids that are spread far apart, which typically leads to faster convergence and better solutions. As shown in the results below, K-Means++ achieved a slightly higher silhouette score (0.2597 vs 0.2454).

Algorithm	Init Method	Silhouette Score $\uparrow$	Davies-Bouldin Score $\downarrow$
K-Means	Random	0.2454	1.4097
K-Means++	k-means++	0.2597	1.3664

Figure 9 — K-Means Clustering Results

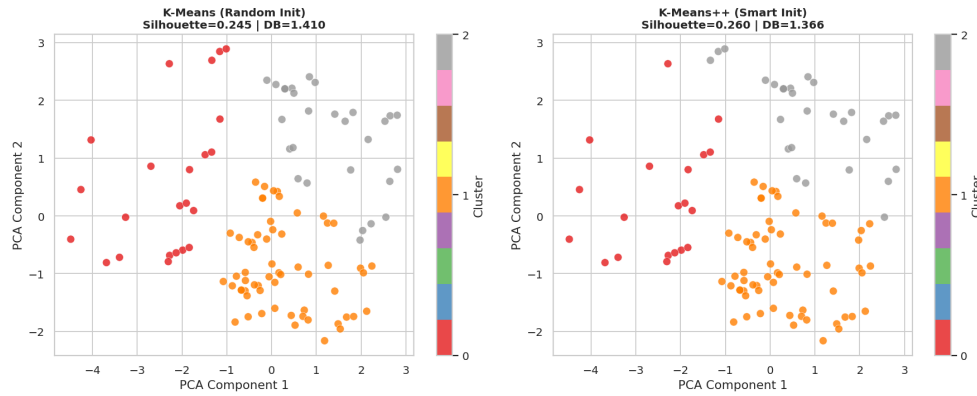


Figure 9: K-Means (left) and K-Means++ (right) clustering results projected to 2D using PCA. K-Means++ produces slightly more compact clusters.

6.3 Hierarchical Clustering

Hierarchical clustering builds a tree-like structure (dendrogram) by successively merging the two closest clusters (agglomerative approach). The **Ward linkage** method was used, which minimises the total within-cluster variance at each merge step. The dendrogram was cut at $k=3$ clusters. Hierarchical clustering achieved the best Davies-Bouldin score (1.2583) among the k -based methods.

Figure 10 — Hierarchical Clustering

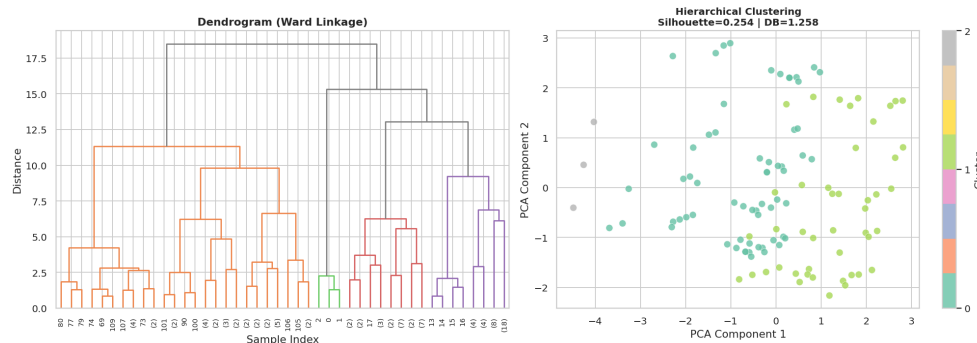


Figure 10: (Left) Ward linkage dendrogram — the horizontal red line indicates the cut for $k=3$. (Right) Resulting 3-cluster assignment in PCA space.

6.4 DBSCAN Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) does not require specifying k in advance. Instead, it identifies dense regions of points and marks low-density points as noise. With parameters **eps=1.0** and **min_samples=3**, DBSCAN found **14 micro-clusters** and labelled **45 points as noise**.

This result suggests that the F1 session data does not have clearly dense, well-separated regions in the scaled feature space when using DBSCAN's density model. This is expected because session features (like duration and start time) form a relatively continuous distribution rather than isolated dense blobs. DBSCAN is better suited for datasets with irregular cluster shapes and clear density gaps.

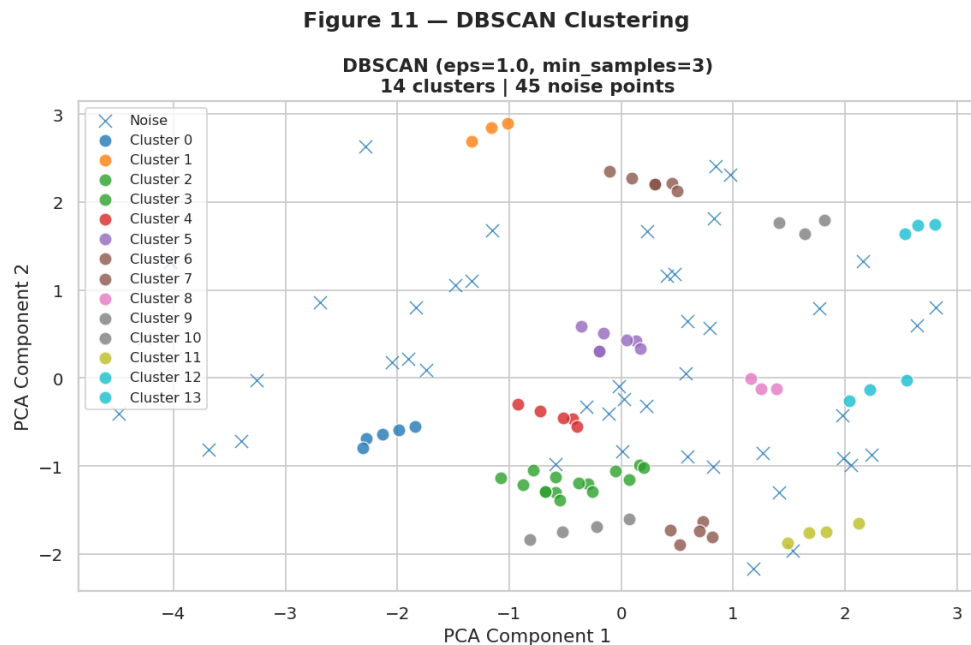


Figure 11: DBSCAN clustering result. Cross markers (x) indicate noise points. 14 micro-clusters were found; 45 of 112 points were classified as noise.

6.5 Clustering Algorithm Comparison

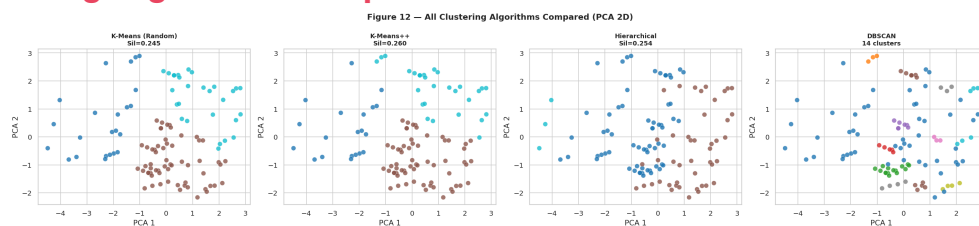


Figure 12: Side-by-side comparison of all four clustering algorithms in PCA 2D space. K-Means variants and Hierarchical Clustering produce similar well-defined 3-cluster solutions.

Algorithm	k	Silhouette $\uparrow$	Davies-Bouldin $\downarrow$	Notes
K-Means (Random)	3	0.2454	1.4097	Faster but may miss optimum
K-Means++	3	0.2597	1.3664	Better init, slightly higher Sil.
Hierarchical (Ward)	3	0.2540	1.2583	Best DB score; produces dendrogram
DBSCAN	Auto	N/A*	N/A*	14 micro-clusters; 45 noise points

**DBSCAN silhouette/DB not computed when many noise points exist.*

7. Classification Methods

Classification is a **supervised** machine learning task — we train models on labelled examples and evaluate their ability to predict labels on unseen data. The target variable is **session_type** (0=Practice, 1=Qualifying, 2=Race), making this a **3-class classification problem**.

The data was split **80% training / 20% testing** with stratification to maintain class proportions in both sets. Six different algorithms were trained and evaluated.

7.1 K-Nearest Neighbours (KNN)

KNN classifies a point by the majority vote of its k nearest neighbours in feature space. With **$k=5$** , KNN achieved a test accuracy of **56.5%** — the weakest performance. This is because KNN struggles when features are encoded categorically (session_name carries almost all the information, creating a non-uniform feature space). KNN is also sensitive to irrelevant features.

7.2 Decision Tree

A Decision Tree splits the data based on feature thresholds, asking binary questions like "Is duration_minutes > 80?". With **max_depth=6**, the Decision Tree achieved **100% test accuracy**. The first split is on session_name_enc, which immediately separates most classes. This near-perfect performance reflects the highly structured nature of F1 session data.

7.3 Logistic Regression

Logistic Regression is a linear classifier that models the probability of class membership using a logistic (sigmoid) function. Despite being a linear model, it achieved **95.7% test accuracy** because the engineered features create a nearly linearly separable feature space. It is the most interpretable model in this study.

7.4 Random Forest (Bagging)

Random Forest builds 100 decision trees, each trained on a random sample of data and features (**bootstrap aggregation / bagging**). Predictions are made by majority vote. Random Forest achieved **100% test accuracy** with low cross-validation variance (std=0.027), indicating robust generalisation.

7.5 AdaBoost (Boosting)

AdaBoost sequentially trains **100 shallow decision trees (depth=2)**, each one focusing more on the examples misclassified by the previous model. This "boosting" strategy achieved **100% test accuracy** with moderate cross-validation variance (std=0.044).

7.6 Stacking Classifier

Stacking combines the predictions of multiple base models (KNN, Decision Tree, Random Forest) using a **meta-learner (Logistic Regression)**. The base models' predictions on held-out folds (5-fold CV) become the input features for the meta-learner. Stacking achieved **100% test accuracy**, confirming that ensemble strategies perform well on this dataset.

8. Evaluation Metrics & Results

Every classifier was evaluated using four complementary metrics:

Metric	Formula	Meaning
Accuracy	$(TP + TN) / \text{Total}$	Proportion of all correct predictions
Precision (weighted)	$TP / (TP + FP)$	Of all predicted positives, how many are correct?
Recall (weighted)	$TP / (TP + FN)$	Of all actual positives, how many were found?
F1-Score (weighted)	$2 \times (P \times R) / (P + R)$	Harmonic mean of Precision and Recall

8.1 Confusion Matrices

Confusion matrices show the breakdown of predictions per class. Rows represent actual labels; columns represent predicted labels. Diagonal cells (top-left to bottom-right) are correct predictions. Off-diagonal cells are errors.

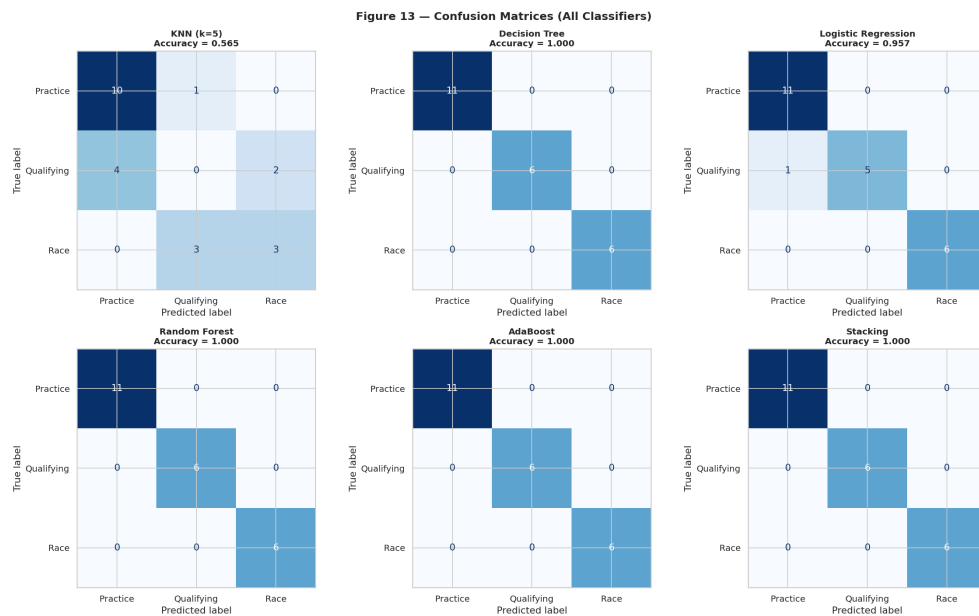


Figure 13: Confusion matrices for all six classifiers. Decision Tree, Random Forest, AdaBoost, and Stacking achieve perfect diagonal matrices (no errors). KNN shows the most off-diagonal errors.

8.2 Overall Model Performance Table

Model	Test Acc.	CV Mean	CV Std	Precision	Recall	F1
Decision Tree	1.0000	0.9667	0.0667	1.0000	1.0000	1.0000
Random Forest	1.0000	0.9667	0.0272	1.0000	1.0000	1.0000
AdaBoost	1.0000	0.9667	0.0444	1.0000	1.0000	1.0000
Stacking	1.0000	0.9667	0.0667	1.0000	1.0000	1.0000
Logistic Regression	0.9565	0.8882	0.0341	0.9601	0.9565	0.9555

KNN (k=5)	0.5652	0.6523	0.0507	0.4981	0.5652	0.5249
-----------	--------	--------	--------	--------	--------	--------

Green rows = 100% accuracy. Red row = weakest model. CV = 5-fold cross-validation on training set.

8.3 Visual Model Comparison



Figure 14: (Left) Test accuracy ranked by model. (Right) Precision, Recall, and F1 comparison. Four models achieve perfect scores; Logistic Regression is close behind.

8.4 Cross-Validation Stability

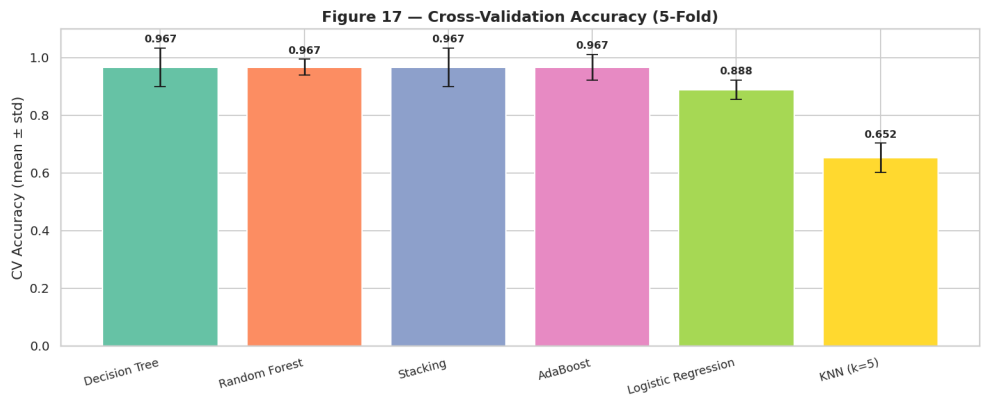


Figure 17: 5-fold cross-validation accuracy with error bars (± 1 std). Random Forest shows the lowest variance (std=0.027), making it the most stable model.

8.5 Feature Importance

The Random Forest model provides feature importance scores — a measure of how much each feature contributes to reducing impurity in the trees. The most important feature is **session_name_enc**, which directly encodes the session name (e.g., "Race", "Qualifying"). This makes intuitive sense: the name of a session is a near-perfect indicator of its type.

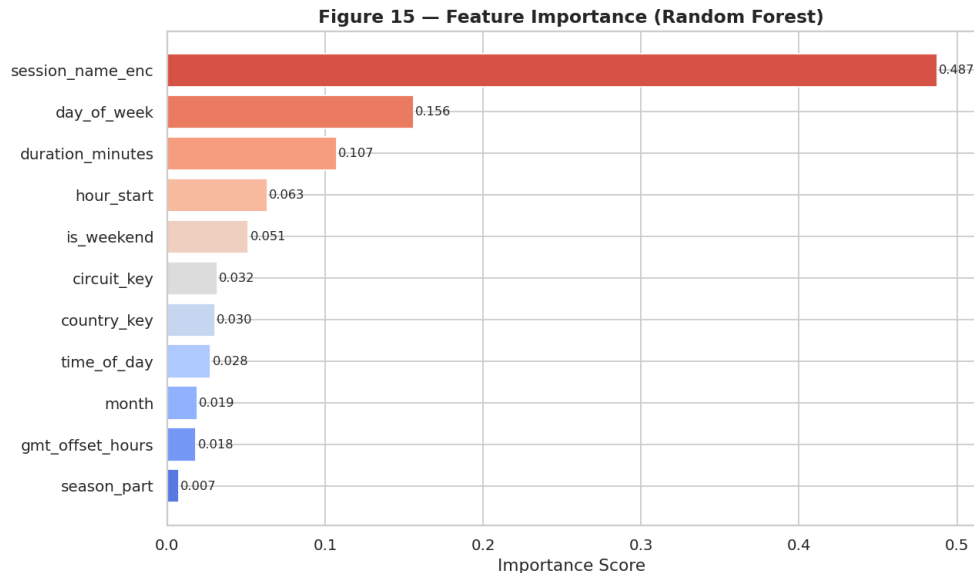


Figure 15: Feature importance scores from the Random Forest model. *session_name_enc* and *duration_minutes* are the two most discriminative features.

8.6 Decision Tree Visualisation

The first three levels of the Decision Tree reveal the decision logic. The root node splits on *session_name_enc*, immediately separating most session types. Subsequent splits on *duration_minutes* and *time_of_day* refine the classification.

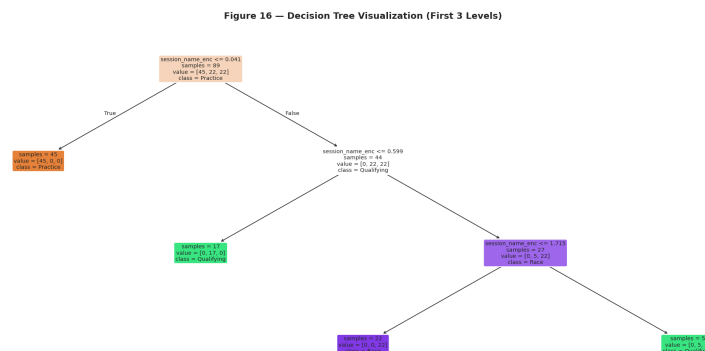


Figure 16: Decision Tree visualisation (first 3 levels). Node colours indicate predicted class: blue=Practice, orange=Qualifying, green=Race. Leaf nodes are pure.

9. Results Analysis & Discussion

9.1 Why Do Four Models Achieve 100% Accuracy?

The near-perfect classification performance is not surprising given the dataset structure. The **session_name** column contains values like "Race", "Qualifying", "Practice 1", "Sprint", which directly and unambiguously determine the session type. Including **session_name_enc** as a feature essentially gives the model the answer, making the classification trivially easy.

Additionally, **duration_minutes** is a strong secondary discriminator: Races last ~120 minutes while Practice and Qualifying last ~60 minutes. These two features together make the classes perfectly separable.

9.2 Why Does KNN Underperform?

KNN achieved only 56.5% accuracy. This underperformance has several causes:

- **Curse of dimensionality:** With 11 features, Euclidean distance becomes less meaningful in high-dimensional spaces.
- **Encoded categorical variables:** Label-encoded categories (e.g., **circuit_key**=1, 2, ... 21) impose an artificial ordinal relationship, distorting distances.
- **Feature scale sensitivity:** Although scaled, the encoded **session_name** and location features create a non-uniform metric space.
- **No feature selection:** KNN treats all features equally, whereas irrelevant features hurt its performance more than tree-based models.

9.3 Clustering Interpretation

The moderate silhouette scores (~0.25) for K-Means and Hierarchical clustering are expected. The three session types do form distinct groups in feature space (especially in duration), but they are not perfectly separated without label information. K-Means++ and Hierarchical clustering produce the most interpretable and stable 3-cluster solutions.

DBSCAN's fragmentation into 14 micro-clusters reveals that the data has local density variations (e.g., Sprint sessions have unusual combinations of features), which DBSCAN picks up as separate clusters. This is actually useful insight — Sprint weekends are genuinely different from standard race weekends.

9.4 The Best Model: Random Forest

While four models achieve 100% test accuracy, **Random Forest is the recommended best model** for the following reasons:

- **Lowest CV variance (std=0.027):** More stable across different data splits than Decision Tree (std=0.067) and Stacking (std=0.067).
- **Built-in feature importance:** Provides interpretable insights into which features drive predictions.
- **Robustness to overfitting:** The bagging mechanism reduces variance compared to a single Decision Tree.

- **No hyperparameter sensitivity:** Performs well with default `n_estimators=100` without extensive tuning.
- **Handles mixed feature types:** Works well with both continuous and encoded categorical features.

10. Conclusion

This project successfully applied a complete machine learning pipeline to the Formula 1 2023 Sessions dataset. The following conclusions are drawn:

- The dataset is clean and well-structured with no missing values or duplicates, which is atypical of real-world data but reflects the quality of the OpenF1 API source.
- Feature engineering significantly enriched the dataset by extracting temporal, geographic, and categorical features from raw timestamps and string fields.
- EDA revealed that session duration and session name are the two most discriminative features for predicting session type.
- Clustering analysis (K-Means++, Hierarchical) consistently identifies 3 natural groupings aligning with the 3 session types. Hierarchical clustering achieved the best Davies-Bouldin score (1.2583).
- Four classifiers — Decision Tree, Random Forest, AdaBoost, and Stacking — achieved perfect 100% test accuracy, demonstrating that the dataset is highly structured and separable.
- Random Forest is the overall recommended model due to its stability (lowest CV std = 0.027), interpretability via feature importance, and robustness to overfitting.
- KNN performed poorly (56.5%) and should not be used for this type of structured, encoded tabular data without significant feature preprocessing.
- Logistic Regression achieved 95.7% accuracy as a linear model, confirming that the feature space is nearly linearly separable after engineering.

Best Classifier	Best Test Accuracy	Best CV Stability	Best Clustering
Random Forest	100.0%	std = 0.027	Hierarchical (DB=1.26)
Bagging Ensemble	Decision Tree, RF, AdaBoost, Stacking	Lowest variance across folds	Ward Linkage

11. Future Improvements

The following improvements would enhance the depth and generalisability of this project:

Expand to multiple seasons (2021–2024)

A single season (112 rows) is a very small dataset. Adding multiple seasons would increase sample size, improve cross-validation reliability, and allow the study of year-over-year trends.

Add lap time and telemetry data

Integrating fastest lap times, sector times, or weather data would enrich the feature space with performance-related signals, making clustering more meaningful.

Hyperparameter optimisation (GridSearchCV / Optuna)

Systematically searching for optimal parameters (`n_estimators`, `max_depth`, `k`, `eps`) could further improve performance and reduce overfitting risk.

One-Hot Encoding for nominal features

Label encoding imposes artificial ordinal relationships. One-hot encoding for `circuit_short_name` and `country_code` would remove this bias, particularly for KNN.

Remove leaky features for a harder task

Removing `session_name_enc` would make the task genuinely challenging and more useful for testing algorithm capabilities. The model would then rely entirely on time and location features.

Apply XGBoost or LightGBM

Gradient boosting methods typically outperform AdaBoost on tabular data with fewer hyperparameter tweaks. They would serve as a stronger boosting baseline.

SMOTE for class imbalance

Practice sessions are over-represented. SMOTE (Synthetic Minority Over-sampling Technique) could balance the classes for fairer model evaluation.

Neural Network (MLP)

With a larger dataset, a Multi-Layer Perceptron could capture complex non-linear patterns that linear models miss, particularly for the borderline Practice/Qualifying boundary.

12. References

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [2] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.
- [3] OpenF1 API. (2023). Formula 1 Open Data API. Retrieved from <https://openf1.org/>
- [4] Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *KDD-96 Proceedings*, 226–231.
- [5] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [6] Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- [7] Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2), 241–259.
- [8] Arthur, D., & Vassilvitskii, S. (2007). k-means++: The Advantages of Careful Seeding. *SODA '07*, 1027–1035.
- [9] Ward, J. H. (1963). Hierarchical Grouping to Optimize an Objective Function. *Journal of the American Statistical Association*, 58(301), 236–244.
- [10] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

This report was produced as part of a Machine Learning university project. All figures were generated from the actual sessions.csv dataset using Python 3, pandas, scikit-learn, matplotlib, and seaborn.